

**;PRAKTIKUM SISTEM CERDAS DAN PENDUKUNG KEPUTUSAN
SEMESTER GENAP TA 2025/2026**

LAPORAN PROYEK AKHIR



DISUSUN OLEH:

NIM	:	123240100 123240109
NAMA	:	Nicolaus Narindra Lianto Muthia Umairah
PLUG	:	IF-J
NAMA ASISTEN	:	Bagas Duta Prasetya (123220057) Nandisya Faiz Effendi (123220139)

**PROGRAM STUDI INFORMATIKA
JURUSAN INFORMATIKA
FAKULTAS TEKNIK INDUSTRI
UNIVERSITAS PEMBANGUNAN NASIONAL "VETERAN"
YOGYAKARTA
2026**

HALAMAN PENGESAHAN

LAPORAN PROYEK AKHIR

Disusun oleh:

Nicolaus Narindra Lianto	123240100
Muthia Umairah	123240109

Telah Diperiksa dan Disetujui oleh Asisten Praktikum Sistem Cerdas dan Pendukung
Keputusan

Pada Tanggal:

Asisten Praktikum

Bagas Duta Prasetya
NIM. 123220057

Asisten Praktikum

Nandisya Faiz Effendi
NIM. 123220139

KATA PENGANTAR

Puji syukur saya panjatkan kepada Tuhan Yang Maha Esa yang senantiasa mencurahkan rahmat dan hidayah-Nya sehingga saya dapat menyelesaikan praktikum Sistem Cerdas dan Pendukung Keputusan serta laporan proyek akhir praktikum yang berjudul Sistem Pemilihan *Electric Vehicle*. Adapun laporan ini berisi tentang proyek akhir yang saya pilih dari hasil pembelajaran selama praktikum berlangsung.

Tidak lupa ucapan terimakasih kepada asisten praktikum yang selalu membimbing dan mengajari saya dalam melaksanakan praktikum dan dalam menyusun laporan ini. Laporan ini masih sangat jauh dari kesempurnaan, oleh karena itu kritik serta saran yang membangun saya harapkan untuk menyempurnakan laporan akhir ini.

Atas perhatian dari semua pihak yang membantu penulisan ini, saya ucapkan terimakasih. Semoga laporan ini dapat dipergunakan seperlunya.

Yogyakarta, 4 Juni 2026

Penyusun 1,

Penyusun 2,

Nicolaus Narindra Lianto

NIM. 123240100

Muthia Umairah

NIM. 123240109

DAFTAR ISI

HALAMAN PENGESAHAN.....	ii
KATA PENGANTAR.....	iii
DAFTAR ISI.....	iv
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang Masalah.....	1
1.2 Tujuan Proyek Akhir.....	1
1.3 Manfaat Proyek Akhir.....	1
BAB II PEMBAHASAN.....	2
2.1 Dasar Teori.....	2
2.2 Skenario Kasus & Dataset.....	2
2.3 Pemodelan Sistem Pendukung Keputusan.....	2
2.4 Perhitungan & Algoritma.....	2
2.5 Implementasi & Tampilan Sistem.....	2
BAB III JADWAL Pengerjaan DAN PEMBAGIAN TUGAS.....	3
3.1 Jadwal Pengerjaan.....	3
3.2 Pembagian Tugas.....	3
BAB IV KESIMPULAN DAN SARAN.....	4
4.2 Kesimpulan.....	4
4.3 Saran.....	4
DAFTAR PUSTAKA.....	5

BAB I

PENDAHULUAN

1.1 Latar Belakang Masalah

Meningkatnya variasi kendaraan listrik (*Electric Vehicle*) di pasaran dengan spesifikasi yang sangat beragam—mulai dari kapasitas jarak tempuh (*range*), efisiensi daya, hingga fitur kenyamanan—sering kali membuat calon pembeli mengalami kesulitan dalam menentukan pilihan terbaik yang sesuai dengan kebutuhan dan anggaran mereka. Banyaknya kriteria kuantitatif dan kualitatif seperti efisiensi energi, akselerasi, kapasitas kargo, hingga kecepatan pengisian daya (*fast charging*) membuat proses evaluasi manual menjadi tidak akurat dan subjektif. Oleh karena itu, diperlukan suatu sistem objektif yang dapat mengolah multi-kriteria tersebut guna memberikan rekomendasi kendaraan listrik yang paling optimal bagi pengguna.

1.2 Tujuan Proyek Akhir

Tujuan dari proyek akhir ini adalah merancang dan membangun sebuah Sistem Pendukung Keputusan berbasis *web* menggunakan metode *Weighted Product* (WP) untuk menentukan peringkat kendaraan listrik terbaik berdasarkan bobot kepentingan kriteria yang diinput oleh pengguna. Melalui sistem ini, data spesifikasi dari berbagai alternatif kendaraan listrik akan dihitung secara matematis menggunakan perkalian matriks keputusan yang telah dipangkatkan dengan bobot kriteria, sehingga menghasilkan rekomendasi keputusan yang akurat, transparan, dan terarah.

1.3 Manfaat Proyek Akhir

Proyek akhir ini diharapkan dapat memberikan manfaat praktis bagi calon pembeli sebagai panduan atau referensi objektif dalam memilih kendaraan listrik yang paling efisien dan sesuai dengan preferensi pribadi mereka. Bagi perkembangan ilmu komputer, proyek ini menjadi implementasi nyata dari integrasi metode *Weighted Product* dalam menyelesaikan masalah pemilihan multi-atribut (*Multi-Attribute Decision Making*) pada industri otomotif modern. Selain itu, sistem ini juga dapat dimanfaatkan oleh dealer atau penyedia layanan untuk membantu memetakan produk yang paling relevan dengan kebutuhan konsumen secara cepat dan tepat.

BAB II PEMBAHASAN

2.1 Dasar Teori

Metode Weighted Product (WP) adalah salah satu pendekatan Multi-Attribute Decision Making (MADM) yang digunakan untuk menentukan alternatif terbaik di antara beberapa pilihan berdasarkan sejumlah kriteria, dengan cara mengalikan nilai kriteria yang telah “dipangkatkan” bobotnya. Dalam konteks pemilihan mobil elektrik, metode ini memungkinkan pengambil keputusan mempertimbangkan berbagai kriteria sekaligus.

Weighted Product mengasumsikan bahwa setiap kriteria bersifat independen dan sudah memiliki bobot yang menunjukkan tingkat kepentingannya. Skor keseluruhan setiap alternatif (misalnya tiap model mobil elektrik) dihitung dengan perkalian nilai kriteria yang dipangkatkan dengan bobot kriteria tersebut, bukan dengan penjumlahan seperti pada metode SAW.

$$W_j = \frac{w_j}{\sum w_j}$$
$$V_i = \prod_{j=1}^n (x_{ij})^{w_j}$$

Secara umum, skor alternatif A_i dirumuskan sebagai;

$$V_i = \prod_{j=1}^n (x_{ij})^{w_j}$$

dengan:

- n : jumlah kriteria.
- x_{ij} : nilai alternatif A_i pada kriteria ke- j ,
- w_j : bobot kriteria ke- j ,
- V_i : nilai preferensi relatif alternatif A_i .

Dalam aplikasi SPK, kriteria biasanya dibagi menjadi:

1. Kriteria keuntungan (benefit): makin tinggi nilainya semakin baik.
2. Kriteria biaya (cost): makin rendah nilainya semakin baik.

Untuk kriteria keuntungan, bobot digunakan sebagai pangkat positif, sedangkan untuk kriteria biaya dapat digunakan pangkat negatif atau dilakukan transformasi nilai agar nilai “semakin besar” tetap berarti “lebih baik”. Setiap bobot umumnya dinormalisasi sedemikian sehingga jumlah seluruh bobot $w_j = 1$.

2.2 Skenario Kasus & Dataset

Masalah yang diangkat dalam sistem ini adalah bagaimana membantu seseorang memilih mobil elektrik terbaik dari banyak alternatif yang tersedia, ketika setiap kendaraan memiliki karakteristik yang berbeda dan pengguna memiliki kriteria impian yang ingin

diprioritaskan. Dalam permasalahan ini, pengambil keputusannya adalah user atau calon pembeli mobil listrik itu sendiri, yaitu orang yang sedang mencari kendaraan elektrik paling sesuai dengan preferensi pribadinya. Misalnya, user adalah seseorang yang ingin membeli mobil elektrik terbaik tanpa batasan budget, tetapi ia tetap memiliki kriteria ideal seperti jarak tempuh yang jauh, efisiensi energi yang baik, akselerasi yang nyaman, kapasitas penumpang yang cukup, serta ruang bagasi yang memadai. Tujuan akhirnya adalah menghasilkan rekomendasi dan perbandingan kendaraan listrik yang paling sesuai dengan kebutuhan dan preferensi user, sehingga user dapat mengambil keputusan secara lebih cepat, objektif, dan terarah berdasarkan data yang tersedia.

Sebagai simulasi, dipakai dataset Kaggle dengan judul “*Electric Vehicle Specs Datasets (2025)*” yang mengumpulkan data spesifikasi dari EV-Database.org, sebuah situs yang menyediakan informasi lengkap tentang mobil listrik di Eropa dan pasar global. Struktur datanya sengaja dirancang rapi dalam format tabel CSV agar mudah dibaca oleh Python (pandas), Excel, atau tools analisis data lain. Dataset mencakup spesifikasi 470+ mobil listrik dari berbagai merek, model, segmen pemasaran, dan negara (terutama pasar Eropa dan beberapa negara lain). Jumlah record bisa bervariasi tergantung pembaruan, tetapi umumnya mencakup mobil listrik all-electric dan beberapa model plug-in hybrid yang relevan.

Dataset berisi spesifikasi kendaraan listrik dari berbagai merek dan model, sehingga dapat menjadi dasar perbandingan antar alternatif. Setiap data kendaraan memuat atribut penting seperti brand, model, range atau jarak tempuh, efficiency, acceleration, charging power, seats, dan cargo volume. Dengan data tersebut, sistem dapat menilai kelebihan dan kekurangan tiap mobil berdasarkan kriteria yang ditentukan user. Secara umum, dataset ini mewakili kumpulan pilihan mobil elektrik di pasar, lalu diproses agar dapat dibandingkan secara adil melalui metode sistem pendukung keputusan, sehingga hasil akhirnya berupa ranking kendaraan yang paling mendekati mobil impian user.

2.3 Pemodelan Sistem Pendukung Keputusan

Dalam dataset yang dipakai, telah tersedia 6 spesifikasi untuk mobil elektrik yang tercatat. Spesifikasi mobil elektrik yang tersedia serta peran sebagai *Cost* dan *Benefit* dalam dataset tersebut adalah sebagai berikut:

1. Rata-rata jarak tempuh (*Benefit*),
2. Efisiensi baterai (*Cost*),
3. Akselerasi mesin (*Cost*),
4. Kapasitas daya baterai (*Benefit*),
5. Jumlah kursi (*Benefit*), dan
6. Kapasitas bagasi (*Benefit*).

Alternatif mobil tersedia dalam berbagai brand mobil elektrik dan model yang bermacam-macam. Total data yang tersedia mencapai lebih dari 470 data yang dapat dibandingkan menggunakan metode Weighted Product.

Pengaturan bobot setiap kriteria dibebaskan sesuai kebutuhan pengguna. Juga pengguna bebas memilih brand mana yang akan dibandingkan model mobilnya agar dapat mempercepat proses pemilihan mobil terbaik sesuai kriteria pengguna.

2.4 Perhitungan & Algoritma

Prosedur umum Weighted Product dalam sistem pendukung keputusan pemilihan mobil elektrik adalah:

1. Menentukan kriteria dan alternatif
2. Memberikan bobot pada setiap kriteria
3. Menentukan matriks keputusan dan menghitung matriks normalisasi
4. Menghitung vektor preferensi V_i
5. Perangkingan dan pemilihan alternatif terbaik

2.5 Implementasi & Tampilan Sistem

(isi dengan source code program)

Source Code Main

```
import streamlit as st

dashboard_page = st.Page("dashboard.py", title="Dashboard")
process_page = st.Page("process.py", title="Process")
profile_page = st.Page("profile.py", title="Profile")

st.set_page_config(page_title="SCPK")
pg = st.navigation({"Tools": [dashboard_page, process_page, profile_page]})
pg.run()
```

Source Code Show All Data

```
import streamlit as st
import pandas as pd
from EVChoose import prepare_data
from EVChoose import criteriaDisplayMap
st.header("Sistem Cerdas Pemilihan Kendaraan Listrik")
st.write("Sistem Pendukung Keputusan (SPK) ini digunakan untuk membantu pengguna "
        "dalam memilih kendaraan listrik (EV) terbaik dari beberapa alternatif " \
        "yang tersedia.")

data_raw = pd.read_csv('clean_ev_spec_dataset.csv')
df_unfiltered = prepare_data()[0]

st.subheader("Data Quality Metrics")
col1, col2, col3 = st.columns(3)

with col1:
    with st.container(border=True):
        st.metric(label="Total Data Asli", value=f"{len(data_raw)} Baris")
```



```

        st.caption("Jumlah total data yang diunggah ke sistem.")

with col2:
    with st.container(border=True):
        st.metric(label="Data Bersih", value=f"{len(df_unfiltered)}
        Baris")
        st.caption("Data siap proses (Nilai kriteria > 0).")

with col3:
    with st.container(border=True):
        jumlah_cacat = len(data_raw) - len(df_unfiltered)
        st.metric(label=" Data Cacat (Nilai 0)", value=f"{jumlah_cacat}
        Baris")
        st.caption("Data otomatis dieliminasi dari perhitungan WP.")

st.divider()

st.subheader("Galeri Eksplorasi Data Mentah")
filter_user = st.multiselect(
    "Pilih brand yang ingin ditampilkan:",
    options=df_unfiltered["brand"].unique(),
    default=[],
    key="brand_filter",
)

df_show = df_unfiltered.copy()

if filter_user:
    df_show = df_unfiltered[df_unfiltered["brand"].isin(filter_user)]

st.write(f"Menampilkan {len(df_show)} data:")
st.dataframe(df_show.rename(columns=criteriaDisplayMap), width="stretch")

```

Source Code Extract Raw Dataset

```

import pandas as pd

dataFrame = pd.read_csv("default_ev_spec_dataset.csv")

newDataFrame = DataFrame[['brand', 'model', 'range_km',
'efficiency_wh_per_km', 'acceleration_0_100_s',
'fast_charging_power_kw_dc', 'seats', 'cargo_volume_l']]

newDataFrame.to_csv("clean_ev_spec_dataset.csv", index=False)

```

Source Code Calculation Process

```

# Import Library yang diperlukan
import pandas as pd
import numpy as np

criteria_map = [
    ("Range", "range_km"),
    ("Efficiency", "efficiency_wh_per_km"),
    ("Acceleration", "acceleration_0_100_s"),
    ("Charging Time", "fast_charging_power_kw_dc"),
    ("Seats", "seats"),

```

```

    ("Cargo Space", "cargo_volume_1"),
]

cost_criteria = {"Efficiency", "Acceleration"}
criteriaLabels = [label for label, _ in criteria_map] #ambil criteria_map [(0, _)]
criteriaColumns = [column for _, column in criteria_map] #ambil criteria_map [(_, 0)]
criteriaDisplayMap = {column: label for label, column in criteria_map}

def hitung_prioritas(sorted_items, cost_criteria_set=None):
    #Perhitungan bobot berdasarkan prioritas
    items = sorted_items[0]['items']
    total_bobot = sum(range(1, len(items) + 1))
    bobot_prioritas = []
    active_cost_criteria = cost_criteria if cost_criteria_set is None
    else set(cost_criteria_set)

    for idx, item in enumerate(items):
        nilai_prioritas = len(items) - idx
        bobot = nilai_prioritas / total_bobot

        if item in active_cost_criteria:
            bobot *= -1

        bobot_prioritas.append({
            "Kriteria": item,
            "Bobot": bobot
        })

    return pd.DataFrame(bobot_prioritas)

# Hitung nilai S
def computeSValue(dataFrame, criteriaColumns, normalizeWeight):
    S = []

    for i in range(len(dataFrame)):
        nilai = 1
        for j, col in enumerate(criteriaColumns):
            nilai *= dataFrame.iloc[i][col] ** normalizeWeight[j]
        S.append(nilai)

    return pd.Series(S, name='S')

# Hitung Nilai V (preferensi)
def computeVValue(dataFrame):
    total_S = dataFrame['S'].sum()
    if total_S == 0:
        raise ValueError("Total nilai S tidak boleh 0")
    valueV = dataFrame['S'] / total_S
    return pd.Series(valueV, name='V')

# Perankingan
def computeRank(dataFrame):
    dataFrame['Rank'] = dataFrame['V'].rank(ascending=False)
    dataFrameFinal = dataFrame.sort_values('V',
ascending=False).reset_index(drop=True)

    return pd.DataFrame(dataFrameFinal)

```

```

def prepare_data(clean_csv='clean_ev_spec_dataset.csv'):
    """Load and clean dataset, return dataframe ready for scoring.

    Returns a dataframe with columns ['brand','model'] + criteriaColumns.
    """
    data = pd.read_csv(clean_csv)

    criteriaColumns = [
        'range_km', #Benefit
        'efficiency_wh_per_km', #Cost
        'acceleration_0_100_s', #Cost
        'fast_charging_power_kw_dc', #Benefit
        'seats', #Benefit
        'cargo_volume_l' #Benefit
    ]

    # Clean data
    dataNumeric = data[criteriaColumns].apply(pd.to_numeric,
errors='coerce')

    dataCleaned = data[dataNumeric.notna().all(axis=1)].copy()
    dataCleaned[criteriaColumns] =
dataNumeric[dataNumeric.notna().all(axis=1)]

    dataCleaned = dataCleaned[(dataCleaned[criteriaColumns] !=
0).all(axis=1)]

    dataframe = dataCleaned[['brand', 'model'] +
criteriaColumns].reset_index(drop=True)
    return dataframe, criteriaColumns

```

Source Code Show Calculation Results

```

import streamlit as st
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from streamlit_extras.dataframe_explorer import dataframe_explorer
from EVChoose import (
    criteriaLabels,
    criteriaColumns,
    cost_criteria,
    prepare_data,
    computeSValue,
    computeVValue,
    computeRank,
    hitung_prioritas,
)
from streamlit_sortables import sort_items

df, criteriaColumns = prepare_data()

if "user_vehicles" not in st.session_state:
    st.session_state.user_vehicles = []

original_items = [

```

```

    {'items': criteriaLabels}
]

# Helper: normalisasi bobot
def _manual_to_prioritas_df(row: dict) -> pd.DataFrame:
    total = sum(row.values()) or 1
    rows = []
    for label in criteriaLabels:
        w = row[label] / total
        if label in cost_criteria:
            w *= -1
        rows.append({"Kriteria": label, "Bobot": w})
    return pd.DataFrame(rows)

def apply_dark_theme(ax, fig):
    fig.patch.set_facecolor("#0E1117")
    ax.set_facecolor("#0E1117")
    ax.tick_params(colors="white")
    ax.xaxis.label.set_color("white")
    ax.yaxis.label.set_color("white")
    ax.title.set_color("white")
    for spine in ax.spines.values():
        spine.set_color("white")
    ax.grid(color="gray", linestyle="--", linewidth=0.5, alpha=0.3)

def get_combined_df(filtered_df, user_vehicles):
    """Gabungkan data sistem dengan data kustom user.
    Kolom '_source' ditambahkan di sini secara terpusat, bukan dari dict
    input.
    """
    base = filtered_df.copy()
    base["_source"] = "system"

    if not user_vehicles:
        return base

    # Buat user_df dari kolom kriteria + brand + model saja (tanpa
    _source)
    user_df = pd.DataFrame(user_vehicles)

    # Pastikan semua kolom kriteria ada, isi None jika ada yang kosong
    for col in (["brand", "model"] + criteriaColumns):
        if col not in user_df.columns:
            user_df[col] = None

    # Tambahkan _source khusus untuk data user
    user_df["_source"] = "user"

    # Samakan urutan kolom dengan base
    user_df = user_df[[c for c in base.columns if c in user_df.columns]]

    # Kolom yang ada di base tapi tidak di user_df → isi NaN
    for col in base.columns:
        if col not in user_df.columns:
            user_df[col] = None

    user_df = user_df[base.columns] # paksa urutan kolom identik

```

```

return pd.concat([base, user_df], ignore_index=True)

simple_style = """
.sortable-component {
    background-color: black;
    font-size: 16px;
    counter-reset: item;
}
.sortable-item {
    background-color: black;
    color: white;
}
"""

st.title("Pemilihan EV Terbaik")
st.empty()

# Filter brand
df['brand'] = df['brand'].astype(str)
brand_options = sorted(df["brand"].dropna().unique())

selected_brands = st.multiselect(
    "Filter brand",
    options=brand_options,
    default=[]
)
df_filter_brand = df.copy()

if selected_brands:
    df_filter_brand = df_filter_brand[df_filter_brand["brand"].isin(selected_brands)]

df_filter_brand = df_filter_brand.reset_index(drop=True)
df_numeric_only = df_filter_brand.select_dtypes(include=['number'])
df_filter_numeric = dataframe_explorer(df_numeric_only, case=False)
mask = df_filter_brand.index.isin(df_filter_numeric.index)
filtered_df = df_filter_brand[mask].reset_index(drop=True)

# Tambah kendaraan kustom
add_custom = st.toggle("Tambah kendaraan sendiri untuk dibandingkan",
value=False)

if add_custom:
    st.subheader("Input kendaraan kustom")

    with st.expander("Tambah kendaraan baru", expanded=True):
        with st.form("form_kendaraan_baru"):
            col1, col2 = st.columns(2)
            with col1:
                custom_brand = st.text_input("Brand")
            with col2:
                custom_model = st.text_input("Model")

            # Input nilai untuk setiap kriteria
            custom_vals = {}
            cols = st.columns(2)
            for i, col in enumerate(criteriaColumns):

```

```

        label = criteriaLabels[i]
        with cols[i % 2]:
            custom_vals[col] = st.number_input(
                label, min_value=0.0, step=0.1,
key=f"custom_{col}"
            )

        submitted = st.form_submit_button("Simpan kendaraan")

        #validasi brand & model wajib diisi
        if submitted:
            if not custom_brand.strip() or not custom_model.strip():
                st.error("Brand dan Model wajib diisi.")
            else:
                #simpan tanpa "_source", ditambahkan nanti di
get_combined_df
                new_vehicle = {
                    "brand": custom_brand.strip(),
                    "model": custom_model.strip(),
                    **custom_vals,
                }
                st.session_state.user_vehicles.append(new_vehicle)
                st.success(f" {custom_brand} {custom_model} berhasil
ditambahkan!")
                st.rerun()

        # Tampilkan & kelola daftar kendaraan user
        if st.session_state.user_vehicles:
            st.write("Kendaraan kustom yang akan ikut dihitung:")
            for i, v in enumerate(st.session_state.user_vehicles):
                col1, col2 = st.columns([5, 1])
                with col1:
                    st.write(f"{v['brand']} - {v['model']}")
                with col2:
                    if st.button("Hapus", key=f"del_{i}"):
                        st.session_state.user_vehicles.pop(i)
                        st.rerun()
        else:
            st.info("Belum ada kendaraan kustom. Tambahkan menggunakan form
di atas.")

        # Pilih metode bobot
        input_method = st.selectbox(
            "Pilih metode input bobot:",
            options=["Drag & Drop Prioritas", "Slider", "Number Input"],
            help="Pilih cara Anda ingin menentukan bobot kriteria"
        )

        with st.form(key="priorities_form", enter_to_submit=True):
            st.write("Atur bobot prioritas kriteria:")

            if input_method == "Drag & Drop Prioritas":
                st.caption("Seret kriteria untuk mengurutkan prioritas (atas =
prioritas tertinggi).")
                sorted_items = sort_items(original_items, multi_containers=True,
custom_style=simple_style)
                slider_weights = None
                number_weights = None

```

```

elif input_method == "Slider":
    st.caption("Geser slider untuk menentukan bobot tiap kriteria
(0-100). Bobot akan dinormalisasi otomatis.")
    slider_weights = {}
    cols = st.columns(2)
    for i, label in enumerate(criteriaLabels):
        with cols[i % 2]:
            slider_weights[label] = st.slider(
                label, min_value=0, max_value=100, value=50, step=1,
                key=f"slider_{label}"
            )
    sorted_items = None
    number_weights = None

else:
    st.caption("Masukkan bobot untuk tiap kriteria (0.0-1.0). Bobot
akan dinormalisasi otomatis.")
    number_weights = {}
    cols = st.columns(2)
    for i, label in enumerate(criteriaLabels):
        with cols[i % 2]:
            number_weights[label] = st.number_input(
                label,
                min_value=0.0, max_value=1.0,
                value=round(1 / len(criteriaLabels), 2),
                step=0.05, format="%.2f",
                key=f"num_{label}"
            )
    sorted_items = None
    slider_weights = None

run_clicked = st.form_submit_button(label="Jalankan Proses SPK")

# Proses SPK
if run_clicked:

    # Validasi bobot
    if input_method == "Drag & Drop Prioritas":
        if sorted_items is None:
            st.error("Gagal membaca urutan prioritas. Silakan coba
lagi.")
        st.stop()
        df_prioritas = hitung_prioritas(sorted_items)

    elif input_method == "Slider":
        if slider_weights is None or sum(slider_weights.values()) == 0:
            st.error("Semua bobot slider bernilai 0. Sesuaikan minimal
satu slider.")
        st.stop()
        df_prioritas = _manual_to_prioritas_df(slider_weights)

    else:
        if number_weights is None or sum(number_weights.values()) == 0:
            st.error("Semua bobot bernilai 0. Masukkan setidaknya satu
bobot.")
        st.stop()
        df_prioritas = _manual_to_prioritas_df(number_weights)

```

```

    bobot_series = df_prioritas.set_index("Kriteria")["Bobot"]
    weights = [bobot_series[label] for label in criteriaLabels]

    # Bangun combined_df, pisahkan _source sebelum dikirim ke fungsi
    hitung
        combined_df = get_combined_df(filtered_df,
st.session_state.user_vehicles)

    # final_df untuk kalkulasi, hanya kolom identitas + kriteria (tanpa
_source)
    # _source disimpan terpisah sebagai series, di-merge kembali setelah
hitung
    source_series = combined_df["_source"].copy() # simpan mapping index
→ source
    final_df = combined_df[["brand", "model"] + criteriaColumns].copy()

    # Tampilkan bobot
    bobot_display = {
        row["Kriteria"]: f"{abs(row['Bobot']) * 100:.2f}%"
        for _, row in df_prioritas.iterrows()
    }

    bobot_df = pd.DataFrame(list(bobot_display.items()),
columns=['Kriteria', 'Bobot'])

    with st.expander("Lihat bobot"):
        st.table(bobot_df)
        fig, ax = plt.subplots(figsize=(8, 4))
        bobot_abs = df_prioritas['Bobot'].abs() * 100
        bars = ax.bar(df_prioritas["Kriteria"], bobot_abs,
color="#FF4B4B")
        ax.bar_label(bars, fmt='%.1f%%', padding=3, label_type='center',
color='white')
        ax.set_title("Bobot Prioritas Kriteria")
        ax.set_ylabel("Persentase (%)")
        apply_dark_theme(ax, fig)
        st.pyplot(fig)
        plt.close(fig)

    if final_df.empty:
        st.warning("Tidak ada data yang memenuhi kriteria filter.")
    else:
        final_df = final_df.reset_index(drop=True)

        # hitung S, final_df sudah bersih dari kolom "_source"
        final_df["S"] = computeSValue(final_df, criteriaColumns, weights)

        # merge _source kembali ke final_df setelah hitung S
        final_df["_source"] = source_series.values

        # helper label sumber yang human-readable
        def source_label(src):
            return "Kustom" if src == "user" else "Sistem"

        # Lihat nilai S
        with st.expander("Lihat nilai S"):
            df_s = (
                final_df[["brand", "model", "_source", "S"]]
                .copy()

```



```

        .sort_values("S", ascending=False)
        .reset_index(drop=True)
    )
    df_s["Sumber"] = df_s["_source"].apply(source_label)
    st.dataframe(
        df_s[["brand", "model", "Sumber",
"S"]].set_index("brand"),
        use_container_width=True
    )

    top_s = df_s.head(10)
    fig, ax = plt.subplots(figsize=(8, 5))
    # warna berbeda untuk data user
    colors_s = ["#FFD700" if s == "user" else "#FF4B4B" for s in
top_s["_source"]]
    ax.plot(top_s["model"], top_s["S"], marker="o", linewidth=2,
color="#FF4B4B")
    ax.set_title("Perbandingan Top 10 Nilai S")
    ax.set_xlabel("Model EV")
    ax.set_ylabel("Nilai S")
    plt.xticks(rotation=30)
    apply_dark_theme(ax, fig)
    st.pyplot(fig)
    plt.close(fig)

# Hitung V
final_df["V"] = computeVValue(final_df)

# Lihat nilai V
with st.expander("Lihat nilai V"):
    df_v = (
        final_df[["brand", "model", "_source", "V"]]
        .copy()
        .sort_values("V", ascending=False)
        .reset_index(drop=True)
    )
    df_v["Sumber"] = df_v["_source"].apply(source_label)
    st.dataframe(
        df_v[["brand", "model", "Sumber",
"V"]].set_index("brand"),
        use_container_width=True
    )

# Perankingan
# computeRank menghasilkan DataFrame baru, re-merge _source
berdasarkan brand+model
finalRank = computeRank(final_df)

# Buat lookup brand+model → _source dari final_df
source_lookup = (
    final_df[["brand", "model", "_source"]]
    .drop_duplicates(subset=["brand", "model"])
    .set_index(["brand", "model"])["_source"]
)

# tambahkan kembali kolom _source ke finalRank
finalRank["_source"] = finalRank.apply(
    lambda r: source_lookup.get((r["brand"], r["model"])),
"system"),

```

```

        axis=1
    )
    finalRank["Sumber"] = finalRank["_source"].apply(source_label)

    # highlight_user pakai kolom "_source" yang sudah benar ada
    def make_highlighter(source_col="_source"):
        def highlight_user(row):
            if row.get(source_col, "system") == "user":
                return ["background-color: #1a3a1a; color: #7fff7f"]
        * len(row)
        return [""] * len(row)
    return highlight_user

top10 = finalRank.head(10).copy()
display_rank = (
    top10
    .astype({"Rank": int})
    [ ["Rank", "brand", "model", "Sumber", "V"] ]
    .set_index("Rank")
)

st.subheader("Top 10 Ranking EV")

# Periksa apakah ada data user di top 10
user_in_top10 = top10[top10["_source"] == "user"]
if not user_in_top10.empty:
    nama_user = ", ".join(
        f"{r['brand']} {r['model']} (Rank #{int(r['Rank'])})"
        for _, r in user_in_top10.iterrows()
    )
    st.success(f"Kendaraan kustom masuk top 10: **{nama_user}**")
else:
    # Cari rank kendaraan user di luar top 10
    user_ranks = finalRank[finalRank["_source"] == "user"]
    if not user_ranks.empty:
        nama_user = ", ".join(
            f"{r['brand']} {r['model']} (Rank #{int(r['Rank'])})"
            for _, r in user_ranks.iterrows()
        )
        st.info(f"Kendaraan kustom: {nama_user}")

    # highlight bekerja karena "_source" ada di display_rank sebelum
    di-drop index
    st.dataframe(
        display_rank.style.apply(make_highlighter(), axis=1),
        use_container_width=True
    )

st.toast("Proses SPK selesai.")

# Grafik ranking
ranking_chart = top10.copy()
fig, ax = plt.subplots(figsize=(8, 5))

# warna bar berbeda untuk data user
bar_colors = ["#FFD700" if s == "user" else "#3B82F6"
              for s in ranking_chart["_source"]]

```

```

        bars = ax.barh(ranking_chart["model"], ranking_chart["V"],
color=bar_colors)
        ax.bar_label(bars, fmt='%.4f', padding=5, label_type='center',
color='white')
        ax.set_title("Top 10 Ranking EV ")
        ax.set_xlabel("Nilai V")
        ax.invert_yaxis()
        apply_dark_theme(ax, fig)
        st.pyplot(fig)
        plt.close(fig)

```

Source Code Profile Developer

```

import streamlit as st

st.set_page_config(layout="wide")

def developer_card(name, role, skills, image_url, bio):
    with st.container(border=True):
        col1, col2 = st.columns([1, 2])

        with col1:
            st.image(image_url, width="stretch")

        with col2:
            st.subheader(name)
            st.caption(f"🚀 {role}")
            st.write(f"***Skills:** {skills}")

        # Fitur detail lebih lanjut menggunakan expander
        with st.expander("Lihat Detail Profil"):
            st.write(bio)
            # st.button("Hubungi Developer", key=name)

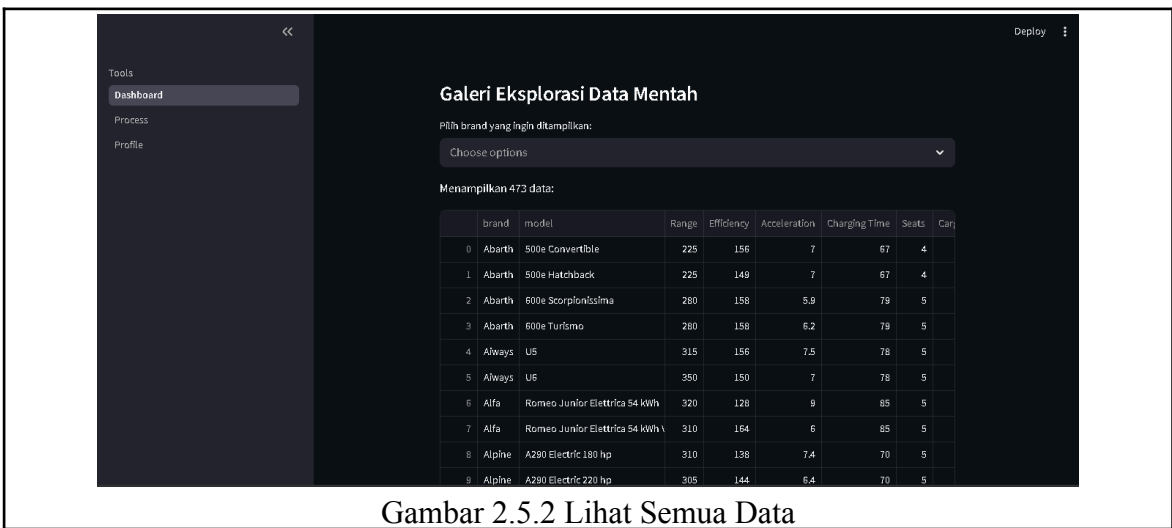
st.title("Team Developer")

left_col, right_col = st.columns(2)

with left_col:
    developer_card(
        name="Nicolaus Narindra Lianto",
        role="Backend Developer",
        skills="ctrl C + ctrl V",
        image_url="https://www.w3schools.com/howto/img_avatar.png",
        bio="Semua deadline tak trabas asal Claude Pro"
    )

with right_col:
    developer_card(
        name="Muthia Umairah",
        role="Frontend Developer",
        skills="Deadline Solver Spek Bondowoso",
        image_url="https://www.w3schools.com/howto/img_avatar2.png",
        bio="Minat tidur dengan tenang 24 jam tanpa beban deadline"
    )

```

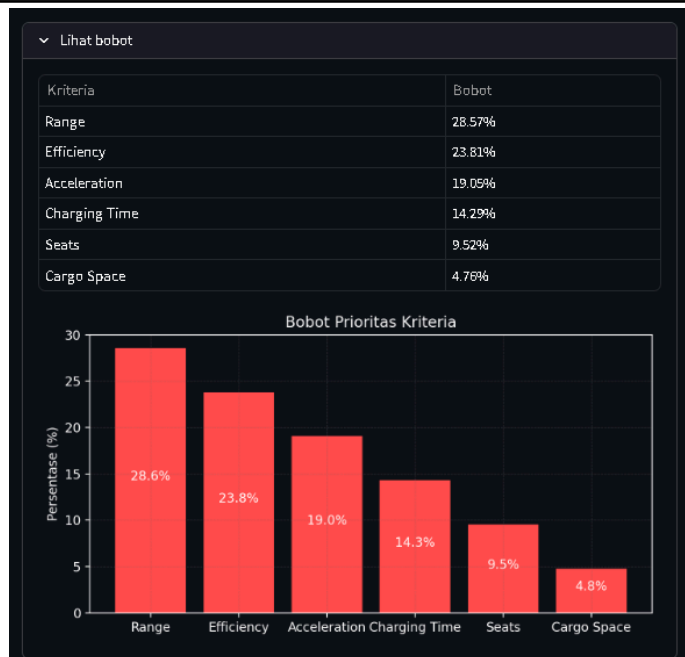




Gambar 2.5.4 Input Data Baru



Gambar 2.5.5 Overview Hasil Perhitungan



Gambar 2.5.6 Tabel Bobot



Gambar 2.5.7 Tabel Nilai S

~ Lihat nilai V

brand	model	Sumber	V
Lucid	Air Grand Touring	Sistem	0.0031
Porsche	Taycan Turbo S	Sistem	0.003
Porsche	Taycan Turbo	Sistem	0.003
Porsche	Taycan Turbo S Sport Turismo	Sistem	0.003
Tesla	Model S Plaid	Sistem	0.003
Porsche	Taycan Turbo GT	Sistem	0.0029
Audi	e-tron GT RS performance	Sistem	0.0029
Porsche	Taycan Turbo Sport Turismo	Sistem	0.0029
Porsche	Taycan Turbo S Cross Turismo	Sistem	0.0029
Lucid	Air Touring	Sistem	0.0029

Gambar 2.5.8 Tabel Nilai V



Gambar 2.5.9 Perankingan Top 10

BAB III

JADWAL Pengerjaan dan Pembagian Tugas

3.1 Jadwal Pengerjaan

No	Kegiatan	2026				
		Minggu				
		1	2	3	4	5
1	Perencanaan Struktur Program					
2	Perencanaan Design UI					
3	Pembuatan Kode Proses WP					
4	Pembuatan Tampilan UI					
5	Penyusunan Laporan Akhir					

3.2 Pembagian Tugas

No	Nama	NIM	Deskripsi Tugas
1	Nicolaus Narindra Lianto	123240100	- Merancang alur program - Membuat logic perhitungan spk
2	Muthia Umairah	123240109	- Membuat Tampilan UI - Menyusun Laporan

BAB IV

KESIMPULAN DAN SARAN

4.2 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa Sistem Pendukung Keputusan (SPK) Pemilihan Electric Vehicle berbasis web berhasil dibangun menggunakan metode Weighted Product (WP) dan framework Streamlit. Sistem ini mampu mengolah data spesifikasi kendaraan listrik secara objektif dari dataset yang mencakup lebih dari 470 alternatif berdasarkan enam kriteria utama, yaitu jarak tempuh, efisiensi baterai, akselerasi, kapasitas daya, jumlah kursi, dan kapasitas bagasi. Melalui perhitungan matematis vektor preferensi (Nilai S dan V), sistem sukses memberikan rekomendasi peringkat 10 besar (Top 10) kendaraan listrik yang paling mendekati preferensi dan kebutuhan unik masing-masing pengguna secara akurat, transparan, serta interaktif.

4.3 Saran

Untuk pengembangan sistem lebih lanjut agar dapat menjangkau dan digunakan oleh lebih banyak pengguna lain secara luas, terdapat beberapa hal membangun yang disarankan. Pertama, sistem perlu didukung oleh basis data yang dinamis langsung dari penyedia informasi kendaraan listrik terkini, sehingga data spesifikasi dan harga mobil yang ditampilkan selalu diperbarui tanpa perlu mengunggah file CSV manual secara berkala. Kedua, perlu ditambahkan fitur komparasi harga yang disesuaikan dengan skema pajak lokal atau subsidi pemerintah setempat, mengingat faktor biaya sering menjadi pertimbangan utama bagi masyarakat umum. Terakhir, guna meningkatkan aksesibilitas dan jumlah pengguna, antarmuka sistem harus dioptimalkan menjadi responsif agar nyaman diakses melalui perangkat seluler (mobile-friendly), serta dilengkapi dengan fitur share hasil rekomendasi ke media sosial dalam bentuk grafis atau dokumen PDF agar pengguna dapat membagikan pilihan kendaraan listrik impian mereka dengan mudah.

DAFTAR PUSTAKA

- Hillary Dorothea Kristianto, Universitas Multimedia Nusantara. (n.d.). *Rancang bangun sistem – Bab 2: Landasan teori*. Diakses dari https://kc.umn.ac.id/id/eprint/25587/11/BAB_II.pdf
- Fauziah Alifa. (2017). *BAB II Landasan teori*. Skripsi, Sekolah Tinggi Manajemen Informatika dan Komputer BSI. Diakses dari <https://repository.bsi.ac.id/index.php/unduh/item/291674/FILE10-BAB-2.PDF>
- Kusmarini. (2006). *Weighted Product method dalam sistem pendukung keputusan*. Dalam Konteks MADM / MCDA (rujukan turunan dari Yoon & Hwang).
- Kusumadewi, S. (2013). *Sistem pendukung keputusan multi kriteria*. (Dirujuk dalam beberapa karya ilmiah sebagai sumber teori MADM/Weighted Product).